

Design and Engineering of the ADAM Web Platform for Antimicrobial Peptide Research

ADAM — Antimicrobial Peptide Database Portal

National Taiwan Ocean University · 6 October 2026

Abstract

ADAM is a web platform that gives researchers access to a curated antimicrobial peptide (AMP) database together with sequence-based activity prediction. This paper describes the design and engineering of its front end: the application architecture, how pages are loaded, how shared interface logic is consolidated, how long-running predictions report progress, and how results and uncertainty are communicated to users who are not machine-learning specialists. We also document a set of interface defects found during development. Several of these did not merely affect appearance; they actively withheld information from users, which we argue makes them correctness defects rather than cosmetic ones.

1. Introduction

A prediction model is only useful to a researcher if they can reach it, give it input without friction, understand what it returned, and judge how far to trust it. The front end is where each of those succeeds or fails. This paper therefore treats the interface as part of the scientific instrument rather than as presentation layered on top of one.

Three goals guided the work: keep the existing visual identity unchanged, remove everything that slowed users down or misled them, and make the platform's limitations visible rather than hidden.

2. System architecture

The platform has three tiers. A React single-page application, built with Vite and routed with React Router, runs in the browser. A Node/Express server provides the database and search API. A persistent Python (Flask) service holds the prediction models in memory and performs inference.

Table 1. Responsibilities of each tier.

Tier	Technology	Responsibility
Client	React, Vite, React Router	Pages, input validation, results display
API server	Node.js, Express	Database queries, search, live statistics, routing to models
Inference service	Python, Flask	Keeps models loaded; runs predictions

The inference service runs as a long-lived process deliberately. Loading a protein language model takes far longer than running one prediction, so reloading it per request would make every prediction slow. Keeping it resident means users pay that cost once, at server start-up.

3. Loading performance

3.1 Route-level code splitting

Only the landing page is included in the initial download. Every other page — search, clustering, the guide, structure and sequence search, and each prediction page — is split into its own bundle and loaded only when the user navigates to it. A first-time visitor therefore does not download the whole application before seeing anything.

While a page bundle loads, the placeholder is painted in the site's own background colour, so the transition reads as a brief pause rather than a white flash between header and footer.

3.2 Font loading

The page previously requested its typefaces through thirteen separate stylesheet imports, each of which the browser had to discover and fetch in sequence. These were replaced with a single pre-connected request, removing a chain of blocking round trips from every page load.

4. Shared interface components

4.1 One prediction page, configured per model

Each prediction model originally had its own page, implemented as a near-identical copy of roughly 308 lines. Any change had to be repeated in every copy, and the copies had already begun to drift apart. They were replaced by a single shared component; each model page now supplies only a short configuration describing what differs — its name, endpoint, example input and explanatory text.

Table 2. Effect of consolidating the prediction pages.

Measure	Before	After
Lines per model page	~308	28
Shared implementation	none	1 component
Files edited to change prediction behaviour	one per model	1

4.2 One scroll-reveal behaviour

Seven pages each contained their own copy of the code that fades content in as it scrolls into view. These were replaced with one shared hook. The shared version is also more robust: it reveals content already on screen immediately, reveals content the user has scrolled past, and includes a time-based fallback, so content can never remain permanently hidden if the browser fails to report visibility.

5. The prediction workflow

5.1 Input validation

Submitted sequences are checked before they are sent. Characters that are not standard amino acids are identified and reported to the user, rather than being passed to a model that would silently discard them or return a misleading result. Multi-sequence FASTA input is split into individual records client-side.

5.2 Reporting real progress

A multi-sequence submission to a language model can take minutes. An indeterminate spinner gives the user no basis for deciding whether to wait. Sequences are therefore submitted one at a time, so that each completed response is genuine evidence of progress rather than an animation.

- A progress bar shows completed sequences against the total.
- Remaining time is extrapolated from measured elapsed time, not from a fixed guess, so the estimate corrects itself as the run proceeds.
- A Stop control cancels a run in flight.
- On multi-sequence runs the view keeps the progress bar in sight.
- Animation is disabled for users who have asked their system for reduced motion.

5.3 Model selection

The prediction overview presents each model as a card stating its input, output, accuracy and typical response time, so users can choose between a fast model and a slower, more accurate one with the trade-off in front of them. Users can also run all models at once and receive a combined result.

5.4 Communicating uncertainty

The combined-result page states the weight given to each model and the basis for it, and explicitly tells users that a combined score near 50% means the models disagree and the call is genuinely uncertain. Presenting a single combined number without exposing how it was formed would make it look more authoritative than its parts justify.

Confidence meters are shown only for models that output a probability. A model that outputs an unbounded score is shown as a score, because drawing it as a percentage-style bar would imply a meaning the number does not have.

6. Presenting peptide records

The peptide record page was rebuilt around the sequence itself, which is what a researcher came for. Residues are displayed in a monospaced face grouped in blocks of ten, so positions can be counted by eye, alongside the net charge. A one-click action copies the record in FASTA format, and a second carries the sequence straight into a prediction page, removing a copy-and-paste step.

Cross-references to the Protein Data Bank, UniProt and PubMed became live links. Fields with no data are collapsed behind a toggle instead of occupying space as rows of dashes. Together these reduced the page height from 2,536 px to 1,514 px with no loss of information.

7. Accuracy of displayed information

A recurring finding was that figures shown to users had drifted away from the data behind them. The database size on the search page was hard-coded text, and the landing page understated the number of entries severalfold. These figures are now computed by the server from the loaded dataset and served from a statistics endpoint, so they cannot fall out of date when the database changes.

We treat this as a correctness issue. A research platform that misstates its own coverage gives users a wrong basis for deciding whether a search result is likely to be exhaustive.

8. Defects identified and corrected

The following defects were found during development. The right-hand column describes what each one cost the user.

Table 3. Interface defects and their effect on users.

Defect	Effect on the user
Colour variable referenced but never defined	Invisible search icon; white text on a white badge
Database size written as fixed text	An incorrect figure published
Base style declared after the mobile rule overriding it	Navigation menu unreachable on phones
Grid columns sized 1fr instead of minmax(0, 1fr)	Record details overflowed their container
Unescaped commas in exported data	Corrupted CSV downloads
Scroll reveal with no fallback	Content permanently invisible if visibility was never reported
Import paths with inconsistent letter case	Build would fail on Linux servers
Export columns written out per model by hand	Export broke silently when models were renamed

The last of these is worth singling out. When the prediction system was restructured, the export code still referred to model names that no longer existed and would have produced empty columns without any error. The columns are now generated from the same list of models the page displays, so the two cannot disagree again.

9. Accessibility and compatibility

- Keyboard focus is shown with a visible outline for keyboard users, without adding outlines on mouse clicks.
- Motion is reduced or removed throughout for users who request it at system level.
- All layouts work at phone width without horizontal scrolling, including the navigation menu.
- When pages were renamed during the prediction restructure, the previous addresses were kept working, so bookmarks and links already shared do not break.

10. Conclusion

The ADAM front end was reworked to load faster, to share one implementation where several had drifted apart, to report progress honestly during slow predictions, and to present results and uncertainty in a form a non-specialist can act on. The most important lesson was that interface defects are not always cosmetic: an undefined colour hid a control, fixed text misreported the database, and a hand-written export broke silently. Each of these withheld information from users, and each was fixed at its cause so that it cannot recur.

Note: this document covers the web platform only. The prediction models — dataset, training, evaluation and results — will be added as a separate section once the ESMC FLM training run has completed.